

Adaptive Learning Web

Interview Brief for SDE and DS/ML Discussions

Project Snapshot

- Adaptive Learning Web is a plain Django learning platform for DSA practice. It tracks users, questions, attempts, and topic-wise skill state, then recommends a next question based on learner progress.
- The current deploy-friendly version runs without heavy ML dependencies by default. The ML recommender is optional and can be enabled with `ENABLE_ML_RECOMMENDER=1` when model artifacts and ML packages are available.

Core Architecture

- core: Django project settings, URL routing, WSGI/ASGI entry points.
- users: learner registration, login, profile views, and User ORM model.
- questions: question bank, next-question recommendation view, LeetCode CSV import command, and Question ORM model.
- attempts: attempt logging form, skill update logic, and Attempt ORM model.
- learning: dashboard, progress, knowledge state, retraining entry point, and KnowledgeState ORM model.
- templates/static: server-rendered HTML with template inheritance and a shared CSS file.

Database and ORM

- Django ORM models map to PostgreSQL tables: users, questions, attempts, and knowledge_states.
- Foreign keys connect attempts to users and questions; knowledge states connect a user to a topic and skill score.
- Migrations create and update schema. You should not manually create tables with SQL for normal development.
- Local development can use local PostgreSQL through .env. Render production uses DATABASE_URL.

DSA Question Loading

- The dataset notebook/leetcode_dataset - lc.csv contains title, description, difficulty, related_topics, and url columns.
- A management command now imports or updates questions through the Django ORM: `python manage.py import_leetcode_questions --limit 300`

- After import, verify URL coverage with: `python manage.py shell -c "from questions.models import Question; print(Question.objects.count(), Question.objects.exclude(leetcode_url__isnull=True).exclude(leetcode_url='').count())"`
- The Django admin question list now displays the `leetcode_url` field so you can visually confirm links are present.

Recommendation Flow

- The fallback recommender selects a question from the learner's weakest topic when available, otherwise the first question.
- The optional ML recommender loads `artifacts/dkt_model/dkt_model.keras` and `artifacts/rl_policy/rl_policy.pkl`.
- DKT estimates a user's knowledge state from attempt history; the RL policy scores candidate questions and picks the best next problem.
- Render defaults to fallback mode to avoid TensorFlow build issues. Enable ML only after deploying model artifacts and ML dependencies.

Deployment on Render

- Use build command: `./build.sh`
- Use start command: `gunicorn core.wsgi:application`
- Set `PYTHON_VERSION=3.12.5`, `SECRET_KEY`, and `DATABASE_URL` in Render environment variables.
- Use the external PostgreSQL URL if the database is in a different Render account, and include `sslmode=require`.
- Static files are served through WhiteNoise; `collectstatic` runs during build.

SDE Interview Talking Points

- Explain why the project uses standard Django apps instead of a monolithic API file: separation of concerns and maintainability.
- Describe the ORM relationships and how migrations keep schema changes reproducible.
- Discuss the fallback recommender as graceful degradation: the product still works when ML artifacts are unavailable.
- Mention production readiness improvements: environment variables, Gunicorn, WhiteNoise, Render PostgreSQL, and no secrets in code.
- Call out tradeoffs: server-rendered Django templates are simpler for this project, while a future React frontend could consume JSON views.

DS/ML Interview Talking Points

- The DKT model represents learner knowledge as a sequence model over past attempts.

- Input sequences encode question identity and correctness, allowing the model to infer topic mastery trends.
- The RL policy treats question selection as a decision problem and rewards appropriate challenge levels.
- The system combines interpretable rule-based updates to KnowledgeState with optional model-based recommendation.
- Production ML concern: TensorFlow is heavy on small hosts, so model loading is made optional and isolated behind an environment flag.

Pre-Demo Checklist

Area	What to verify
Question data	Run <code>import_leetcode_questions</code> and confirm <code>leetcode_url</code> count is greater than zero.
Local DB	PostgreSQL service is running and <code>.env</code> has <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_HOST</code> , <code>DB_PORT</code> .
Render DB	<code>DATABASE_URL</code> uses the Render external URL with <code>sslmode=require</code> .
ML mode	<code>ENABLE_ML_RECOMMENDER</code> is unset/0 unless TensorFlow, Keras, NumPy, artifacts, and URLs are de

Command Reference

- Load questions: `python manage.py import_leetcode_questions --limit 300`
- Verify URLs: `python manage.py shell -c "from questions.models import Question; print(Question.objects.count(), Question.objects.exclude(leetcode_url__isnull=True).exclude(leetcode_url='').count())"`
- Run locally: `python manage.py runserver`
- Render deploy: build command `./build.sh`, start command `gunicorn core.wsgi:application`